

Séance 3 - Organisation du code & bonnes pratiques

- **Objectifs et compétences :**
 - Structurer un projet Python (modules, packages).
 - Respecter les conventions PEP8.
 - Documenter avec `docstring` et `typing`.
 - Ajouter des logs pour tracer l'exécution.
- **Modalités d'animation :**
 - Théorie : architecture modulaire, lisibilité et maintenabilité.
 - Démo : refactorisation d'un script monolithique en package.
 - Exercice pratique : créer un package documenté et loggé.
- **Évaluation :** projet restructuré, propre et documenté.
- **Matériel :** Python, éditeur de code, outils de style (`flake8`, `black`).

1 Introduction théorique

Pourquoi structurer un projet ?

- Lisibilité : on comprend rapidement l'organisation.
- Réutilisabilité : chaque module peut être importé ailleurs.
- Maintenabilité : plus facile à corriger et à faire évoluer.
- Collaboration : standards communs dans une équipe.

Modules & Packages

- **Module** = fichier `.py`.
- **Package** = dossier contenant un fichier `__init__.py` et un ou plusieurs modules.

Normes PEP8

- Espaces autour des opérateurs.
 - Indentation en 4 espaces.
 - Noms explicites (`snake_case` pour variables/fonctions, `CamelCase` pour classes).
 - Longueur max : 79 caractères par ligne.
-

2 Mise en place de l'environnement

Avec `uv` pour isoler le projet :

```
uv venv .venv
source .venv/bin/activate # Linux/macOS
.venv\\Scripts\\Activate.ps1 # Windows
```

```
# Installation d'outils de style
uv pip install black flake8
```

3 Exemple : refactorisation d'un script monolithique

Avant (script monolithique `app.py`)

```
import requests

def main():
    url = "<https://jsonplaceholder.typicode.com/posts/1>"
    response = requests.get(url)
    print(response.json())

if __name__ == "__main__":
    main()
```

Après (organisation en package `myapp/`)

```
myapp/
├── __init__.py
├── main.py
└── api.py
```

`api.py`

```
import requests
from typing import Dict

def fetch_post(post_id: int) -> Dict:
    """Récupère un post par son ID depuis l'API JSONPlaceholder."""
    url = f"<https://jsonplaceholder.typicode.com/posts/{post_id}>"
    response = requests.get(url)
    response.raise_for_status()
    return response.json()
```

`main.py`

```
import logging
from .api import fetch_post

logging.basicConfig(level=logging.INFO)

def main():
    logging.info("Démarrage du programme")
    post = fetch_post(1)
    logging.info("Post récupéré avec succès")
    print(post)

if __name__ == "__main__":
    main()
```

4 Documentation avec docstrings & typing

Exemple

```
def add(a: int, b: int) -> int:
    """
    Additionne deux entiers.

    Args:
        a (int): premier nombre
        b (int): deuxième nombre

    Returns:
        int: somme des deux nombres
    """
    return a + b
```

5 Gestion des logs

- **print()** : rapide, mais pas configurable.
- **logging** : adapté aux vrais projets.

Exemple :

```
import logging

logging.basicConfig(
    level=logging.DEBUG,
    format="%(asctime)s - %(levelname)s - %(message)s"
)

logging.info("Info message")
logging.warning("Alerte")
logging.error("Erreur détectée")
```

6 Exercices pratiques

1. Refactorisation

- Prendre un script unique `script.py`.
- Le transformer en package avec plusieurs modules.

2. Documentation

- Ajouter des `docstrings` et du `typing` à toutes les fonctions.

3. Logs

- Remplacer tous les `print()` par `logging`.
- Tester différents niveaux de log.

4. Style

- Vérifier le code avec `flake8`.
- Reformater avec `black`.

